

1. How would you represent the input of the network (vector, matrix, multiple inputs etc.)
2. How would you design the output layer
3. Which activation functions would you use
4. Which Loss function
5. Which (possible) weight regularizers, initializers, etc.
6. On which hyperparameters would you perform the model selection and why
7. How to evaluate the test performance

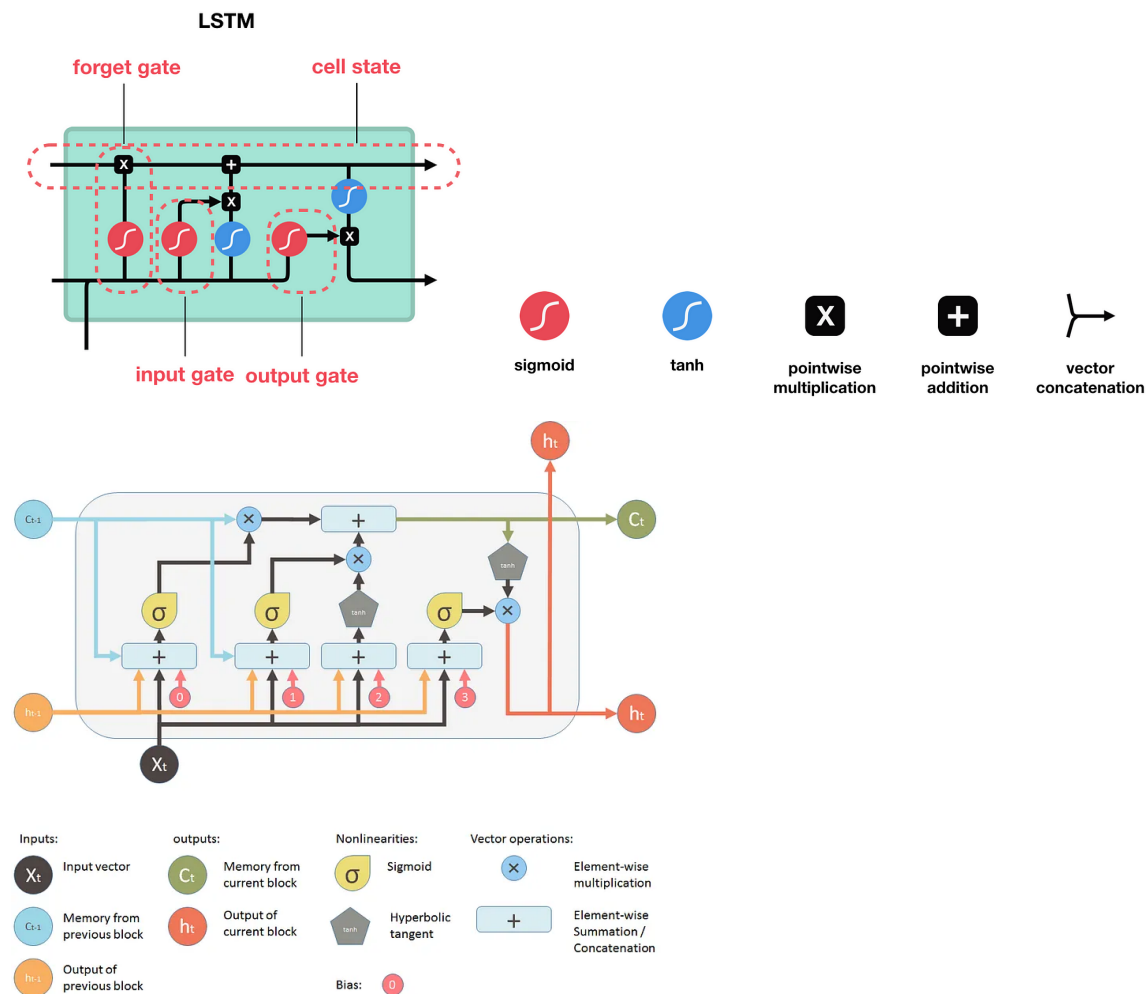
Steps to do

- 1) Performing cleaning
- 2) Create a dictionary of words with keys as integer values
- 3) Prepare the dataset as input and output sets using a dictionary
- 4) Define the LSTM model for text generation

Introduction

- Which kind of task
 - Supervised/unsupervised
 - Character/word generation
- What we have to predict
- *"Find best architecture that is the least complex possible"*
- *"find best hyperparameters that allows the model to perform good both on training and on test"*
- *"the model should be a trade off between computational complexity and accuracy"*
- *"avoid overfitting"*
- Goal: predict the next character of word in the sequence
- LSTM is preferred over RNN because
 - RNN suffers from the vanishing and exploding gradient problem
 - LSTM more suitable to memorize a large amount of previous data
 - With RNN when we add new information it completely modifies the existing information. It is not able to distinguish between important or not so important information
 - **(NOTE:** GRU can be preferable to LSTM, depending on the complexity of the task, since it is much faster, even though this comes at the expense of some loss in the accuracy in the model. In general: choose GRU if the task and the dataset is not too complex and/or you have limited computational resources)
- Takes a sequence of words as input and outputs a matrix of probability to be the next word of the sentence for each word from the dictionary

LSTM



- Gate: decide which data is important and can be useful in the future and which data has to be erased
 - **Forget gate:** which information is important and should be stored and which information we should forget. Takes two inputs: output generated by the previous cell + input of the current cell. The sigmoid activation function produce a value between 0 and 1 which decided which information to keep
 - **Input gate:** add information to the neuron cell, it creates an array of information that has to be added.
 - **Output gate:** select important information from the current cell and show it as output.

How would you represent the input of the network (vector, matrix, multiple inputs etc.)

- Input are word sequences
- Need to find a way on how to create the embedding of a word

- How to represent the word as a vector

Preprocessing

- Lowercase the text
- Split the text at white spaces
- Replace “-” with a white space so that we can split the word better
- Remove short messages (less than 3 words/characters)
- Remove punctuations

Also it would be possible to remove stopwords (NLTK list) and apply stemming (Snowball stemmer) that return the stem of a word (*singing* → *sings*) but this is not suggested since we will use some context and create ambiguity. But this is a trade off and some advantages could be to reduce the complexity and the vocabulary size.

Convert messages in a computer readable form (create a vocabulary)

- Two processes [Text Generation Using LSTM](#)
 - One hot encoding
 - Word embedding

Create a dictionary and one hot encoding

- Count how many unique words are in all the messages
- Create a dictionary with all the unique words where at each word is assigned a unique index (integer) - it could be in alphabetical order
- Create a one-hot encoder to represent each word as a binary vector of 0's and 1. Each vector is of the same length of the vocabulary and the element corresponding to the word's index is set to 1 and the rest to 0.

Target Representation (predict next character):

The target for each input character is the next character in the sequence. So, for the input "Shakespear," the target character would be "e." The target will also be represented as a one-hot vector.

LSTM Architecture:

The LSTM network will take the one-hot encoded input sequence as input. Since LSTM units require a fixed-size input, we need to choose an appropriate sequence length (e.g., 50 characters). We'll feed this fixed-length input sequence into the LSTM layer.

An embedding layer can be used to represent the one-hot encoded characters as continuous dense vectors of lower dimension. This embedding layer can help the model learn more meaningful representations of the characters.

- **Input dim:** size of the vocabulary in the text data. Number of unique words/characters

- **Output dim:** the size of the vector space in which words will be embedded. The size you want to obtain for each word/character
- **Input length:** length of input sequences. How long is your one-hot vector

After the LSTM layer processes the input sequence, the output will be a hidden state, which can be connected to a dense (fully connected) layer with a softmax activation function. The dense layer will predict the probability distribution over the words/characters in the vocabulary for the next word/character in the sequence, and so the number of neurons will be equal to the number of words/characters in the vocabulary.

How would you design the output layer

- Allows to predict the next character in the sequence
- Provide a probability distribution over the characters in the vocabulary for the next character in the sequence
- It is a dense (fully connected) layer followed by a softmax activation function
- Number of neurons equal to the number of characters (if we predict the next character) or equal to the number of unique words (if we are predicting the next word)
- Softmax activation function
 - $f(z) = \frac{e^z}{\sum_{i=1}^N e^{z_i}}$
 - Single output: value between 0 and 1
 - All the outputs: summed up are = 1

Which activation functions would you use

Inside LSTM/GRU

- **Sigmoid:** (output normalized between 0 and 1 + suffer from vanishing gradient problem) $\sigma(z) = \frac{1}{1+e^{-z}}$ + graph
- **Tanh:** (zero-centered, so easier to optimize, but suffer from vanishing gradient problem) $f(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$
- **Forget gate:** use sigmoid to decide what information we're going to throw away from the cell state. A 1 represents "completely keep this" while a 0 represents "completely get rid of this."
- **Input gate:** decide what new information we're going to store in the cell state. Decide which values we'll update (with a sigmoid). Next, a tanh layer creates a vector of new candidate values that could be added to the state. In the next step, we'll combine these two to create an update to the state.
- **Output gate:** push the values with tanh between -1 and 1

Which Loss functions

- Different loss function, depends on how labels are represented and on the task we are dealing with:
 - Regression task with many outliers $\Rightarrow$ **mean absolute error**, since it is less sensitive to outliers (since it isn't squared, the outliers don't have too big weights)
 - Regression task $\Rightarrow$ **mean square error**. Compute the average square difference between predicted values and true values + formula
 - Binary classification $\Rightarrow$ **binary cross entropy loss**. Measure dissimilarity between predicted and true binary labels + formula
 - Multiclass with output as one-hot vector $\Rightarrow$ **categorical cross entropy loss**
 - Multiclass with output as integer $\Rightarrow$ **sparse categorical cross entropy loss**
 - **Square hinge loss** $\Rightarrow$ converge faster and better performance + more robust to noise in the training set labeling + quadratically increase with the error ($\rightarrow$ larger errors are punished more heavily)
- If we have a character (or word) prediction task, we can train the model via (categorical) cross-entropy loss, comparing the one-hot encoding of the character/word with the probability vector output of the model

Which (possible) weight regularizers, initializers, etc.

Regularizer

- Weight regularization
 - Add to the loss function of the model a cost associated with having large weights
 - Use to mitigate overfitting
 - Put constraint on the model
 - Force weight to take small values
 - Make distribution of weight more regular
- **L1**: cost added proportionally to the absolute value of the weight coefficients. Encourage the model to reduce the less important weights to zero + introduce sparsity in the weights by forcing more weights to be zero
- **L2**: cost added proportional to the square of the value of the weights coefficient. Penalize large weights + encourage weights to be small but do not drive them to exactly 0. More used for smaller deep learning models
- **Dropout**: randomly dropping out neurons
 - The output is set to 0
 - No weight update
 - The idea is to introduce noise in the output values to break patterns
 - Learn more robust and general representations

Initializers

- Initialize **weights at random** can lead to:
 - Vanishing gradients: weights and gradients close to 0. Convergence to minimize the loss are very slow 😞
 - Exploding gradients: weights and gradients greater than 1. Impossible to find the optimal minimum 😞
- Initialize weights with random uniform or normal distribution (mean = 0 and standard deviation = 1)
- Xavier/Glorot initialization (tanh or sigmoid)
 - Goal: keep variance (and variance of each gradient) equal for each layer
 - Takes into account the number of inputs and outputs of a layer
 - Two types *uniform* and *normal*
 - Uniform: take the values of the weights with the same probability from the range $\left[-\sqrt{\frac{6}{input + output}} ; \sqrt{\frac{6}{input + output}}\right]$
 - Normal: take the values of the weights from a normal distribution with mean = 0 and standard deviation $\sigma = \sqrt{\frac{2}{input + output}}$
- He initialization (ReLU)
 - Small variation of Xavier initialization
 - Initialize weights from normal distribution
 - Scale weights proportionally to the number of inputs to the layer
 - Two types *uniform* and *normal*
 - Uniform: take the values of the weights with the same probability from the range $\left[-\sqrt{\frac{6}{input}} ; \sqrt{\frac{6}{input}}\right]$
 - Normal: the weights values are taken from a normal distribution with mean 0 and standard deviation equal to $\sigma = \sqrt{\frac{2}{input}}$

Normalizer

- Batch normalization
 - Good idea to apply in a hidden layer *before* the activation function + put the bias of the layer to false
 - Address the **internal covariance shift** that makes training more difficult and when network is trained the parameters change
 - Idea: normalize the inputs of a layer (mean = 0 and variance = 0.1)
 - Reduce dependence of each layer on the specific distribution of inputs
 - Alleviate vanishing and exploding gradients
 - Reduce the need for other regularization techniques
 - Makes the network more robust to changed
 - Two parameters
 - **Gamma**: learned scaling factor (initialized as 1)
 - **Beta**: learned offset factor (initialized as 0)
 - Eliminate the need for dropout
 - Allows to use higher learning rate and to be less careful about initialization

- Acts as a regularizer
- Feature normalization
 - Preprocessing step
 - Applied to input data before feeding it into a machine
 - MinMaxScaling + formula

On which hyperparameters would you perform the model selection and why

- Learning rate: determine the step size at each iteration
 - How quickly or slowly the model learns from the training data
 - Common to start with a moderate learning rate in the range 0.1 to 0.001
- Batch size:
 - Number of sample processed before the model is updated
 - Divide the training dataset into smaller subsets
 - Each mini-batch is a subside of training examples
 - Update the parameters based on the batch
- Epochs: number of times the entire dataset is passed in the model
- Number of layers
- Number of neurons
- Weight initialization
 - Random uniform or normal distribution
 - Xavier/Glorot initialization (Tanh/Sigmoid)
 - He initialization (ReLU)
- Optimizer: update the weights of the neural network during training
 - Stochastic gradient descent (SGD)
 - SGD with momentum
 - Adam
 - RMSprop
 - Adagard

How to evaluate the test performance

- **Perplexity**: measure used to evaluate the performance of language models. It refers to how well the model is able to predict the next word in a sequence of words. It's based on the concept of entropy, which is the amount of chaos or randomness in a system. So a lower perplexity score indicates that the language model is better at calculating the next word that is likely to occur in a given sequence, while a higher perplexity score indicates that the model is less accurate.
- BLEU
- ROUGE

- During the training part we can set a validation set that gives an estimate of model skill while tuning model's hyperparameters.
- Perform a manual search of the parameters that are difficult to tune in a grid search
- We apply a grid search to find some hyperparameters (like the activation function of the hidden layers). Perform on a small range of hyperparameters (very computationally expensive)